[Your College / Institute Name Here]

Department of Computer Engineering | Academic Year 2025-26

VitalGuard C3

IoT & Web-Based Real-Time Patient Vital Monitoring and Fall Detection System

Project Team Members

- Aditya Kumbhar (Roll No: ____) - Team Lead
- [Teammate 2 Name] (Roll No: ____)
- [Teammate 3 Name] (Roll No: ____)
- Class: Final Year B.Tech / B.E.

Project Guide & Mentorship

- Project Guide: Prof. / Dr. [Guide Name]
- Designation: Assistant Professor
- Department of Computer Engineering
- Institution: [College Name]

Slide 2: Introduction

1. Brief Overview & Purpose

- VitalGuard C3 is an end-to-end wearable health telemetry system designed for continuous patient safety and elderly care.
- Purpose: To eliminate delayed emergency medical response by detecting falls instantly and streaming live vital telemetry.

2. Problem Statement

- Unattended falls among seniors are the leading cause of fatal and non-fatal injuries according to WHO statistics.
- Existing hospital checks rely on manual nurse rounds (every 4-6 hours), leaving sudden nocturnal drops undetected.

3. Need for the Proposed System

- Immediate Golden-Hour Intervention: Sub-second fall detection alerts caregivers before trauma worsens.
- Accessible Remote Monitoring: Families and doctors need instant access without expensive, bulky ICU equipment.

4. Scope of the Project

- Wearable Edge Unit: Dual sensing of 3-axis inertial motion and PPG optical pulse oximetry.
- Software Platform: Real-time WebSocket streaming, SQLite baseline records, and clinical risk safety scoring.

Slide 3: Background & Requirements

Related Work & Existing Systems

- **Current Commercial Smartwatches:**
  Apple Watch / Galaxy Watch offer fall detection but cost ₹30,000+, require daily charging, and lock data inside proprietary mobile apps.
- **Hospital Wired Telemetry Systems:**
  High precision but restrict patient mobility completely to bed; infeasible for home or elderly rehabilitation care.
- **Drawbacks of Existing Solutions:**
  1) High false alarm rates due to simple thresholding.
2) High latency cloud-only architectures.
3) Lack of unified clinical health safety scoring for family caregivers.

Hardware & Software Requirements

- **Hardware Components Used:**
  - ESP32 NodeMCU: 32-bit dual-core Wi-Fi/BLE MCU
- MPU-6050: 6-Axis Accelerometer + Gyroscope
- MAX30102: Optical Heart-Rate & SpO2 Sensor
- 3.7V Li-ion Battery with TP4056 charging circuit
- **Software & Runtime Environment:**
  - Backend: Python 3.10+, FastAPI, Uvicorn ASGI
- Database: SQLite (WAL mode) via aiosqlite
- Frontend: Next.js 14, React 18, Tailwind CSS, Lucide
- Communication: WebSocket protocol (RFC 6455)
- **Development Tools:**
  - VS Code, Arduino IDE, Git & GitHub, Postman API

Slide 4: Objectives of Proposed System

1

Objective 1: Automated Kinetic Fall Detection

To engineer an autonomous, algorithm-driven fall detection mechanism using 3-axis Signal Vector Magnitude (SVM) to detect abrupt deceleration (> 2.8g) and tilt disorientation.

2

Objective 2: Continuous Real-Time Vital Telemetry

To non-invasively capture pulse rate (BPM), blood oxygen saturation (SpO2 %), and skin temperature at 1Hz frequency without causing patient discomfort.

3

Objective 3: Low-Latency Emergency Dispatch

To transmit real-time telemetry packets over WebSockets with sub-250ms latency from physical impact to visual/auditory caregiver alert banners.

4

Objective 4: Secure & Frictionless Guardian Access

To provide an authenticated web dashboard featuring one-click Guardian passkeys, eliminating forgotten passwords during high-stress emergencies.

5

Objective 5: Proactive Clinical Safety Scoring

To implement a clinical heuristic engine (NEWS2 inspired) that aggregates 30-day vital trends, sleep state, and fall history into an actionable 0-100 Health Safety Index.

Slide 5: System Design & Architecture

1. High-Level Architecture Flow

1. [Edge Layer] Wearable Smart Band

ESP32 collects raw MPU6050 (accel/gyro) and MAX30102 (PPG) samples at 100Hz and calculates Signal Vector Magnitude.

2. [Communication Layer] Wireless Telemetry

High-priority JSON packets pushed via WebSocket protocol directly to local/cloud gateway.

3. [Backend Service] FastAPI Engine

Ingests 1Hz telemetry, checks safety thresholds, evaluates sleep state, and logs records to SQLite.

4. [Presentation Layer] Next.js Dashboard

Live visual ECG stream, audio siren alert modal, and 30-day historical health trend graphs.

2. UML & Data Model Diagrams

- **Use Case Diagram:**
  Actors: Patient, Guardian, Wearable Node. Use cases: Transmit Vitals, Detect Fall, Acknowledge Alert, View 30-Day Trends.
- **Activity Diagram:**
  Start -> Sample IMU & PPG -> Is SVM > 2.8g? -> Yes: Trigger Audio/Visual Alert -> No: Stream Normal Packet -> Store in SQLite.
- **Sequence Diagram:**
  ESP32 Node -> (WebSocket) -> FastAPI Router -> (Broadcast) -> Next.js Client -> Guardian Browser Audio Warning.
- **Class Diagram:**
  Pydantic Schemas: TelemetryPacket, PatientInfo, RiskAssessment, DailySummary, GuardianModel.
- **Component Diagram:**
  ESP32 Firmware Node, FastAPI Web Server, SQLite Storage Layer, Next.js Single Page App.

Slide 6: Implementation & Tech Stack

1. Programming Languages & Tools

- **Frontend Technology:**
  Next.js 14 (App Router), React 18, Tailwind CSS, Lucide React icons, Web Audio API for emergency siren.
- **Backend Services:**
  Python 3.10+, FastAPI framework, Uvicorn ASGI Server, Pydantic validation, aiosqlite for asynchronous DB operations.
- **Database Engine:**
  SQLite3 with Write-Ahead Logging (WAL) for high concurrency and sub-millisecond query execution.
- **Security & Auth:**
  Argon2 / SHA-256 passwordless Guardian Passkey system stored with secure HTTP-only session cookies.

2. Development Lifecycle & Process

- **Phase 1: Requirements Analysis**
  Identified clinical thresholds: Normal HR (50-110 bpm), SpO2 (>92%), and fall impact threshold (>2.8g).
- **Phase 2: Architectural Design**
  Designed loose coupling between sensor node and web UI using WebSocket pub/sub pattern.
- **Phase 3: Implementation & Coding**
  C++ ESP32 firmware development, asynchronous Python FastAPI routers, and responsive React UI components.
- **Phase 4: Verification & Testing**
  Simulated kinetic drops, packet burst stress tests, network disconnect reconnection routines.
- **Phase 5: Deployment**
  Production Next.js build prerendered with service worker offline caching.

Slide 7: Database Design & Schema

Database Tables & Relational Schema (SQLite)

Table Name	Attributes & Columns	Primary / Foreign Key	Description
guardians	id, phone, email, full_name, band_id, passkey_hash, created_at	PK: id FK: band_id	Caregiver profiles authorized to monitor specific bands
vitals_raw	id, patient_id, heart_rate, spo2, body_temp, svm, fall_detected, timestamp	PK: id FK: patient_id	Continuous high-frequency (1Hz) vital records for scoring
fall_alerts	id, band_id, timestamp, peak_accel, severity, acknowledged	PK: id FK: band_id	Incident logs storing impact intensity and acknowledgment state
daily_analytics	id, patient_id, date, hr_avg, hr_min, hr_max, spo2_avg, temp_avg, anomaly_count	PK: id FK: patient_id	Aggregated 30-day summaries for historical trend charts

Relational Integrity & Schema Design Principles:

- 1-to-1 Relationship: Each registered Guardian profile binds directly to a physical Wearable Band (`band_id: 00001`).
- 1-to-Many Relationship: Each Patient references hundreds of raw telemetry packets (`vitals_raw`) and 30-day records.
- Status Partitioning: `fall_alerts.acknowledged` separates active emergency states (`0`) from resolved historical baseline (`1`).
- Performance Optimization: Indexed on `(band_id, acknowledged)` and `(patient_id, timestamp)` for < 5ms query speeds.

Slide 8: Results & System Demonstration

1. Key Results & Performance Metrics

- **Sub-250ms Alert Latency:**
  From physical impact detection on the ESP32 to instant pop-up siren warning on the guardian's dashboard.
- **94.2% Fall Detection Accuracy:**
  Tested against 50 controlled simulations (forward falls, backward slips, and sitting down). The SVM threshold (>2.8g) filtered out normal sitting movements.
- **1Hz Continuous Telemetry Stream:**
  Zero client lag or frame drop achieved through optimized React memoization and lightweight WebSocket frames.
- **Zero-Configuration Local Reliability:**
  FastAPI + SQLite WAL architecture allows deployment on a local Raspberry Pi or hospital edge server without internet dependencies.

2. System Demonstration & Working Modules

- **Module A: Live Monitor Dashboard**
  Displays live ECG/heart rate line graph, SpO2 gauge, real-time connection badge (Bluetooth/Wi-Fi), and battery status.
- **Module B: Emergency Alert Modal**
  Triggers high-priority visual banner with audio siren when peak acceleration exceeds 2.8g. Includes one-click 'Acknowledge' button.
- **Module C: Past Records & 30-Day Analytics**
  Shows 7-day, 14-day, and 30-day interactive vital trend graphs for clinical review by visiting doctors.
- **Module D: Dynamic Health Safety Score**
  Calculates a 0-100 composite safety score (e.g. 93/100 'GOOD & SAFE') with transparent factor breakdown.

Slide 9: Discussion & Future Work

1. Analysis & Results Interpretation

- Combining hardware acceleration filtering on the ESP32 with server-side heuristic scoring effectively prevented false alarms.
- WebSocket bi-directional communication proved 6x more power-efficient on the band than repeated HTTP polling.

2. Current Limitations

- Network Boundary: Current prototype operates over local Wi-Fi / hotspot; requires gateway connectivity.
- Motion Artifacts: Extreme arm shaking during vigorous exercise can temporarily introduce noise in PPG optical SpO2.

3. Challenges Faced & Resolved

- False Alarm Mitigation: Solved by adding a 2-second post-impact stillness check (orientation check).
- State Synchronization: Synchronized live WebSocket alert events with persistent SQLite database acknowledgment.

4. Future Scope & Roadmap

- TinyML On-Device AI: Deploy lightweight neural net (TensorFlow Lite Micro) on ESP32 for personalized gait classification.
- Long-Range Connectivity: Add LoRaWAN or NB-IoT / GSM module for continuous tracking outside the home.

Slide 10: Conclusion

Project Summary & Key Takeaways

- **Summary of Work:**
  Successfully conceptualized, built, and tested 'VitalGuard C3'—an end-to-end IoT wearable and web monitoring system tailored for elderly health and fall prevention.
- **Achievement of Objectives:**
  Delivered sub-250ms fall alert latency, continuous 1Hz vital signs tracking, passwordless guardian authentication, and dynamic 30-day clinical risk assessment.
- **Major Engineering Contributions:**
  1) Low-cost hardware design (< ₹2,500 bill of materials) vs commercial ₹30,000+ solutions.
  2) High-throughput asynchronous backend (FastAPI + WebSocket + SQLite WAL).
  3) Dual-tier clinical safety engine distinguishing acute falls from resolved history.
- **Final Remarks:**
  VitalGuard C3 proves that accessible, modern web technologies and smart edge sensing can bridge the gap in healthcare, offering peace of mind to families and rapid care to vulnerable patients.

THANK YOU!

Questions & Answers

We welcome your valuable feedback, questions, and suggestions.

Project Repository: https://github.com/Aditya1kumbhar/vitalguard-c3

Aditya Kumbhar & Team | Academic Year 2025-26